

1. What is Java? Write the features of Java.

Java is a high-level, object-oriented programming language developed by Sun Microsystems (now owned by Oracle) in 1995. It is widely used for building cross-platform applications, including web applications, mobile applications, desktop software, and enterprise systems.

Features of Java:

Simple: Java is easy to learn and understand, with a syntax similar to C++ but without complexities like pointers.

Object-Oriented: Everything in Java is treated as an object, allowing modularity and reusability of code.

Platform-Independent: Java follows the "Write Once, Run Anywhere" (WORA) principle, meaning compiled Java code can run on any platform with a Java Virtual Machine (JVM).

Secure: Java provides built-in security features such as bytecode verification and a security manager.

Multithreaded: Java allows concurrent execution of multiple threads for better performance.

Portable: Java code is platform-independent and can be executed on any system with a compatible JVM.

Robust: Java has strong memory management and exception-handling features, making it less prone to crashes.

2. Why is Java a platform-independent language?

Java achieves platform independence through its compilation and execution process:

Compilation into Bytecode

Java source code (.java file) is compiled by the Java compiler (javac) into bytecode (.class file), an intermediate representation of the code.

This bytecode is not tied to any specific operating system.

Execution by the JVM

The bytecode is executed by the Java Virtual Machine (JVM), which interprets it and converts it into machine-specific code at runtime.

Since JVM implementations exist for different platforms (Windows, Linux, macOS), the same bytecode can be executed on any OS with a compatible JVM.

Abstraction from OS and Hardware

Unlike languages like C or C++, which compile into platform-dependent machine code, Java's bytecode remains the same across all platforms.

The JVM ensures that Java programs run consistently on different systems, making it platform-independent.

3. Can Java be said to be a complete object-oriented programming language?

No, Java is not a purely object-oriented language because it supports primitive data types (such as int, char, float, etc.), which are not objects. A fully object-oriented language would treat everything, including primitive data types, as objects.

4. What do you understand by an instance variable and a local variable?

Instance Variable:

Declared inside a class but outside any method.

These variables belong to an instance of a class and are accessed using objects.

Example:

```
class Example {  
    int instanceVar = 10; // Instance variable  
}
```

Local Variable:

Declared inside a method and is only accessible within that method.

It cannot be accessed outside the method.

Example:

```
class Example {  
    void method() {  
        int localVar = 20; // Local variable  
    }  
}
```

5. Explain OOP concepts and write the pillars of OOP with examples.

OOP Concepts:

Object: A real-world entity with attributes and behaviors.

Class: A blueprint for creating objects.

Inheritance: Allows a class to inherit properties from another class.

Polymorphism: Allows a method to have different forms.

Encapsulation: Restricts direct access to data by using private variables and getter/setter methods.

Abstraction: Hides implementation details and shows only relevant features.

Four Pillars of OOP:

Encapsulation:

```
class Person {  
    private String name;  
    public void setName(String name) { this.name = name; }  
    public String getName() { return name; }  
}
```

Inheritance:

```
class Animal { void makeSound() { System.out.println("Animal makes sound"); } }  
class Dog extends Animal { void bark() { System.out.println("Dog barks"); } }
```

Polymorphism:

```
class Shape { void draw() { System.out.println("Drawing a shape"); } }  
class Circle extends Shape { void draw() { System.out.println("Drawing a circle"); } }
```

Abstraction:

```
abstract class Vehicle { abstract void start(); }  
class Car extends Vehicle { void start() { System.out.println("Car starts"); } }
```

6. How is an infinite loop declared in Java with examples?

An infinite loop is a loop that never terminates. Examples:

Using while loop:

```
while (true) {  
    System.out.println("Infinite Loop");  
}
```

Using for loop:

```
for (;;) {  
    System.out.println("Infinite Loop");  
}
```

7. Difference between JVM, JRE, and JDK.

JVM (Java Virtual Machine)

Executes Java bytecode.

JRE (Java Runtime Environment)

Includes JVM and libraries needed to run Java applications.

JDK (Java Development Kit)

Includes JRE, compiler (javac), and development tools.

8. Explain public static void main(String args[]) in Java.

public: Allows access from anywhere.

static: Allows calling the method without creating an object.

void: No return value.

main: Entry point of Java programs.

String args[]: Accepts command-line arguments.

9. Explain different data types in Java.

Primitive Data Types: int, char, boolean, float, double, long, short, byte

Non-Primitive Data Types: String, Array, Class, Interface

10. What are operators and types of operators?

Arithmetic: +, -, *, /, %

Relational: ==, !=, >, <

Logical: &&, ||, !

Bitwise: &, |, ^

11. Assignment: =, +=, -= 11. Write a Java program to generate the Fibonacci series.

```

public class Fibonacci {
    public static void
        main(String[] args) { int
            n = 10, a = 0, b = 1;
            for (int i = 1; i <= n; i++)
                { System.out.print(a + "
                    ");

                    int sum = a + b;
                    a = b;
                    b = sum;
                }
            }
        }
    }
}

```

12. Write a Java program for checking prime numbers.

```

public class PrimeCheck {
    public static void
        main(String[] args) { int
            num = 29, flag = 0;
            for (int i = 2; i <= num / 2; i++) {
                if (num % i == 0) { flag = 1; break; }
            }
            System.out.println(num + (flag == 0 ? " is Prime" : " is Not Prime"));
        }
    }
}

```

13. Write a Java program for checking palindrome numbers.

```

public class Palindrome {
    public static void
        main(String[] args) {
            int num = 121, rev = 0,
            temp = num; while
            (temp > 0) {
                rev = rev * 10
                + temp % 10;
                temp /= 10;
            }
            System.out.println(num + (num == rev ? " is Palindrome" : " is Not Palindrome"));
        }
    }
}

```

```
}  
}
```

14. Write a Java program for checking factorial numbers.

```
public class Factorial {  
    public static void  
        main(String[] args) { int  
            num = 5, fact = 1;  
            for (int i = 1; i <=  
                num; i++) { fact  
                    *= i;  
                }  
            System.out.println("Factorial of " + num + " is " + fact);  
        }  
    }
```